

Hydra Mobile SDK

for Android & iOS

A native Flutter/Dart client for Cardano Hydra

Public v1.0.0 · MIT · `hydra_client` on pub.dev

github.com/dextrlabs-dev/hydra-mobile-sdk

The problem

- Cardano **Hydra** is the leading L2 for near-instant, low-fee payments.
- Existing tooling (hydrasdk, Mesh, Blaze) is **JS/TS / backend-oriented**.
- **No first-class native Dart/Flutter binding** for `hydra-node`.
- Mobile micropayments, tipping, in-game economies, and PoS had **no idiomatic path** to Hydra.

The solution

A thin, well-typed **Dart binding** for `hydra-node` 2.x (HTTP + WebSocket) plus a runnable **Flutter micropayments app**.

- Reconnecting WebSocket sessions (mobile-grade resilience)
- Sequence sync + dedup on replay
- `HydraHeadFacade` : one object for the whole head lifecycle
- Typed REST wrappers + partial models with a raw-map escape hatch
- Android + iOS + Linux

Architecture

```
App (Flutter)
|  sendInit / sendNewTx / sendClose ...
▼
HydraHeadFacade → ReconnectingHydraSession (WS, capped backoff)
|                ↳ SeqTracker + HydraSyncPolicy (dedupe / refresh)
|                → HydraHttpClient (typed REST: head, snapshots, commit, tx)
HydraSigner (interface)  HydraStateStore (pluggable)
```

Conditional WS factory (io / html / stub) → one codebase across mobile, desktop, web.

What we built

- **hydra_client** — publishable Dart package (MIT), pub.dev **1.0.0**.
- **hydra_demo** — micropayments sample app: L1 commit + two in-head L2 scenarios (Dice, Snake).
- **40 automated tests** (33 library + 7 app); CI builds **Android APK + iOS simulator** every push.
- **Docs site**, final report, performance review, slide deck.

Project journey

Phase	Outcome
Initiation	Landscape review + launch doc
Tech assessment	Feasibility, API contract pinned to hydra-node 2.x
Architecture	System blueprint
Alpha (0.2.0)	Core client, reconnect, sync, facade, demo, CI
Public v1.0.0	pub.dev, docs site, report, deck, external POCs

Performance & reliability

- **L2 micropayments:** near-instant in-head settlement (sub-second to low-seconds), independent of L1.
- **L1 cost** paid once per head (open/close), not per payment.
- **Reconnect:** capped 3 s backoff; `messages` stream survives drops.
- **Reliability:** 40/40 tests green; CI builds both mobile targets every push.

KPI: 2 external-team POCs

Team	Use case	Result
Zodor.io	In-app micropayments	 Successful
Bepay.money	Merchant / P2P payments	 Successful

Both integrated the **public** `hydra_client 1.0.0` via the documented API + `hydra_demo` template — no SDK changes needed.

Release & distribution

- **pub.dev:** `hydra_client` `1.0.0` (semantic versioning, MIT).
- **GitHub Release** `v1.0.0`: Android APK + iOS simulator app attached by CI.
- **Docs site:** dextrlabs-dev.github.io/hydra-mobile-sdk
- Automated pub.dev publishing wired via GitHub Actions (OIDC).

Lessons learned

- **Pin the toolchain early** — Cardano serialization packages gate on Flutter versions.
- **Model loosely, then tighten** — raw-map escape hatch survives Hydra API drift.
- **Reconnect + seq-dedup is essential on mobile.**
- **Build mobile targets in CI**, not just the library.
- **A runnable reference app is the best documentation.**

What's next

- Secure-storage `HydraSigner` backends (Android Keystore / iOS Secure Enclave).
- Full typed models for remaining server outputs.
- Opt-in CI integration job spinning up `hydra/demo` Docker nodes.
- Web transport hardening.

Thank you. · github.com/dextrlabs-dev/hydra-mobile-sdk · pub.dev/packages/hydra_client